基于多种文件格式的三维模型在线浏览平台分析

童　凌
（南华大学 计算机科学与技术学院 软件工程系 湖南 衡阳 421001）

摘　要　针对目前三维模型文件需要使用计算机客户端上特定的三维图形软件才能打开，在网页端浏览不便的问题，提出一种在线预览多种 3D 文件的技术。研究了三维模型文件在浏览器上的显示方法，并实现了多种三维模型文件在浏览器中预览和实现的关键技术，并对文件转换时间和加载时间进行相关测试。实验结果表明，使用该技术实现的平台不仅完成了相应的三维文件格式转换，同时实现了良好的在线浏览效果。

关键词　三维文件　在线预览　文件格式转换　Three.js

中图分类号：TP311.5　**文献标志码**：A

DESIGN AND IMPLEMENTATION OF ONLINE 3-DIMENSION VIEWER SUPPORTING MULTIPLE FILE FORMAT

Tong Ling
(University of South China, Institute of Computer Science, Faculty of Software Engineering, 421001, Hunan, Hengyang, China)

Abstract　3-demension files can be opened currently only in specific design software, which is hard to be viewed in web browsers. To solve the inconvenience of viewing online and transforming file format, the author proposes a technic which can make 3-demension files readable in websites. Besides, the article also explores the methods of displaying 3D model in browser as well as the vital technics of multiple type of file formats transformation. The result demonstrates that the platform can successfully transform 3D file formats within a few seconds as well as presents 3D models online.

Keywords　3-demension files　Online view　File transformation　Three.js

0　引　言

网络三维 (Web 3D) 技术采用模型展示的方法，对展示对象进行三维建模，使用渲染引擎展示具有纵深效果的立体图像。[1]三维模型占用的资源和体积较二维图像相比更为庞大，因此在互联网发展的早期，Web 3D 技术面临着网络带宽、处理器速度等多个因素的制约。随着互联网的蓬勃发展，带宽、存储器容量迅速增长，硬件因素已不再成为 Web3D 发展的瓶颈。目前，许多世界知名厂商都基于 Web3D 标准，推出了自己的 Web3D 显示技术以及解决方案。除此以外，目前 Web3D 文件在领域内格式尚未统一，不同的设计软件生成的文件格式不同，给互联网之间的交流和传输带来巨大负担和成本。本文探讨一种使用 JavaScript 语言实现 3D 文件实时在线预览的技术（三维模型浏览引擎），笔者利用此技术将其运用到实际开发中，实现了三维模型文件的文件转换和在线浏览功能。结果表明，该技术实现的平台能够在几秒内完成 3D 文件的读取与展示和文件转换功能，具有良好的推广前景。

1　3D Web 标准

1.1　三维图形标准

WebGL (Web Graphics Library)是一个用于浏览器中的三维图形标准。WebGL 是一个非常底层的 API,而 OPENGL 的着色语言 GLSL 本身就是一个完整的编程环境。[2]这种绘图标准把 JavaScript 和 OpenGL 结合在一起，主要用于显示、渲染 2D 或 3D 图形。[3]大多数现代浏览器采用并实现了这个标准，因此，1WebGL 的标准化进程得以广泛传播。最新版本的浏览器 Internet Explorer 11、Chrome 和 Firefox 均已支持 WebGL 标准。因此基于 WebGL 技术和浏览器平台，开发人员可以利用浏览器内部的 JavaScript 语言开发出众多 3D 图形应用。[4]

WebGL 解决了目前三维在线浏览的两个缺点：第一，目前大多数 3D web 技术都是由不同公司自行开发的解决方案，文件格式不同，调用方法也不同，因此没有统一的标准。[1]WebGL 通过底层通用的 OpenGL 图形接口实现，可以集成在浏览器中直接调用硬件。因此，可以通过编写文件转换模块的方法，将三维文件完整地转换成其他需要的格式，从而解决了统一标准的问题。

童凌(1994-)，男，浙江丽水人，主要研究方向为软件工程，Web 开发。

第二，在大多数情况下，Web3D 浏览技术都需要插件的支持。如果可以减少插件的使用，那么将会减少用户的使用成本，Web3D 将会得到更加广泛的普及。WebGL 可以通过 HTML5 Canvas 元素作为接口，直接在浏览器内访问相应元素，无需任何浏览器插件支持。从而减少了插件的使用，提高了用户体验。

1.2 三维图形开发瓶颈

目前相关的三维图形开发技术有 Cult 3D、Viewpoint、Java 3D、ShockWave3D 等多种实现[15][13]，其基本情况如下表所示：

表 1 Web3D 技术对比

名称	插件要求	特点
Cult3D	需要	文件传输慢，下载时间长；图像质量不高
Viewpoint	需要	基于 XML，呈现的图像质量不高；兼容性较好
Java3D	不需要	对 CPU 算力要求很高，图像质量高，兼容性好
ShockWave3D	需要	通过实景照片和场景集成软件实现，场景质量高

由表 1 可知，虽然现有的 Web 3D 浏览已有多种技术实现，但或多或少存在某些缺陷，无法形成一个通用的平台。总体看来，制约三维图形在浏览器端传输的瓶颈有三个：

（1）WebGL 在设计时定义了针对硬件底层的图形接口，语言集成度很低，所以 WebGL 标准非常晦涩难懂，且部分代码涉及到计算机图形学知识，对于没有图形学基础的开发人员来说很难使用。

（2）三维图形在 Web 层面一直没有一个合适的展示层模块，用户无法通过合适的渠道对三维模型进行预览，常常需要采用浏览器插件,flash 等第三方软件支持以达到浏览 3D 模型的目的。

（3）Web 3D 数据格式文件多种多样，迄今为止没有一个统一的标准，现在常用的文件格式就有.obj, .stl, .dwg, .vml 等。由于不同文件格式内部存储数据信息的格式不同，技术人员需要了解多个不同文件格式，才能在开发应用软件时做出正确的适配，这极大地增加了三维浏览技术在实现上的难度。

1.3 一种新型开发框架：Three.js

为了更好地推广并开发相关 Web 3D 应用，Mr Doob 设计并开发了 Three.js 框架[6]。Three.js 是基于 WebGL 的一个第三方库，使用 JavaScript 编写。使用该框架，可以创建三维场景，包括摄影机，光影，材质等各种对象。[7]以 WebGL 为支撑的 Three.js 引擎具有以下优势[8]：

（1）无需额外插件。Three.js 可以直接支持 3D 图形的 GPU 硬件加速。

（2）开源：Three.js 框架遵循 MIT 开源协议，可自由复制，分发和修改，因此使用成本很低。

（3）跨平台：基于 JavaScript，所以 Three.js 可以在多种操作系统、多种浏览器上使用。

（4）易用性：Three.js 实现了 WebGL 规范的众多功能，封装了众多三维场景渲染所需的重要工具方法与渲染循环，因此，对计算机图形学不了解的程序员也可以很方便地使用。[9]

因此，基于 WebGL 标准，通过与传统的 web 技术相结合，three.js 可以将数据以三维方式呈现出来[10]。该解决方案不仅具有较好的平台适应性，同时也具有方便后续二次开发的优点。

2 系统设计

2.1 系统原理及核心功能

2.1.1 基本结构

三维模型浏览平台的结构是一种融合了 B/S 和 C/S 的混合体系结构。在现代 Web 浏览器中，可以将架构分为前端和后端两个主要模块。前端的构建使用传统的 HTML、CSS 和 JavaScript。[13]其中 HTML 展示了网站数据信息，CSS 定义了网页的样式与外观，而 JavaScript 负责网页上的 3D 图形展示以及用户与网站之间的交互动作；后端业务主要处理业务逻辑以及相关数据操作。在三维模型浏览平台中，两个核心业务如下：

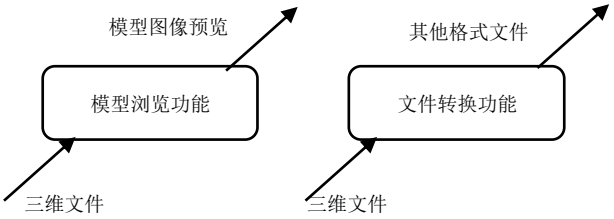

图 1 平台核心业务

2.1.2 模型浏览功能

（1）用户导入三维模型文件后，浏览器发出请求，请求信息被发送到 JavaScript 的图形处理模块。

（2）用 Three.js 编写的图形处理模块接受相关请求，提供各种 3D 文件格式的模型构建功能。

（3）模型构建完毕后，模块生成可供前端展示的代码，交给 WebGL 解析、渲染成 3D 图像，并在浏览器中得以呈现。

2.1.3 文件转换功能：

（1）用户导入文件时，服务器接收文件的数据信息。

（2）根据不同的三维模型数据文件格式，文件转换模块读取三维文件数据源中的点、线、面等基本位置信息和纹理信息，得到该文件构成三维模型的核心数据。

（3）文件转换模块从核心数据中得到物体的点、线、面等核心数据信息，根据各种三维模型的文件格式规则，形成不同的三维格式文件。

（4）服务器将格式文件传输到前端，用户在前端可以下载生成的各种不同格式的三维模型文件。

2.1.4 浏览器显示通用建模步骤

从目前(2016 年)的情况来看，大多数三维模型的呈现方式仍然停留在二维的显示器屏幕上，因此需要讨论三维模型的显示技术。

为了实现 3D 模型显示功能，笔者首先使用 HTML 元素中创造一个<CANVAS>元素，以下所有 JavaScript 操纵的对象均为该元素，使用的浏览器端三维平台搭建方法分析设计如下：

所有的三维模型都有长，宽、高三个属性，将三维模型的相关属性通过图像映射到二维显示器上呈现的过程，就是一个 3D 渲染过程。3D 渲染过程中涉及五个基本步骤，经过以上五个基本步骤，一个三维模型就可以在网页上显示出来，图 1 为三维场景 web 显示的基本结构模块，下面是一个简单的 Three.js 建模步骤以及相关代码：[11]

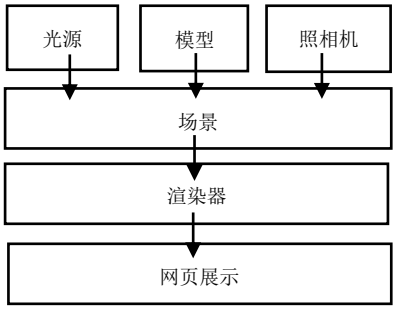

图 2 三维图形显示过程

2.2 场景

场景可以被看作一个在二维平面上“模拟”的三维空间，相当于一个容器。它定义了一个三维坐标系，提供一个参照物，以便确定三维模型的基本数据。后续添加的三维模型都是添加到场景中去。场景的效果会根据观察的距离、视野和角度而发生相应变化。

```
//设置场景
var scene;
function initScene() {
    scene = new THREE.Scene();
}
```

2.3 照相机

它定义了三维空间到二维屏幕的投影方式，从而让三维模型得以呈现。在图形学中，三维场景不可能在一个二维的空间上完全显示，利用 WebGL 可以模拟三维空间，但是最终必须输出在一个二维的显示器上。[12]为了解决这个从三维场景转换到二维平面问题，笔者引入“照相机”的概念。类比于将三维场景转换到一张二维照片上，“照相机”就是这样一个抽象，在建立三维场景时，在某一个位置放置摄像机，并在该视角按照特定的投影方式，得到该视角上的二维投影。

```
  //设置相机
  var cma;
  function initCamera() {
  //照相机按照左手系的位置排布，Y 轴为向上的方向，X 轴位向右的方向，Z 轴是朝内的方向
  // fov 是能看到的视野位置，  aspect 是水平与垂直长度比，near 是相机离物体的最近间距，  far 是照相机距离物体最远的间距。
    cma = new THREE.PerspectiveCamera( 45, height /width, 1, 5000 );
    //设置照相机所在的 x,y,z 上的坐标
    cma.position.x = 0;
    cma.position.y = 50;
    cma.position.z = 100;
    //设置相机的右为「x」轴方向
    cma.right.x = 1;
    cma.up.y = 1;
    cma.up.z = 0;
    cma.lookAt( {x:0, y:0, z:0 } );
  }
```

2.4 形状与材质

在真实的三维场景中，一个物体模型通常有两个属性：几何形状与物体材质。几何形状，如球体、长方体、圆柱、圆锥等存储了几何体的顶点信息。通过大量不同的几何体进行旋转、缩放、并、交、补等运算，即可得到需要组成的 3D 模型；物体材质是独立于物体顶点信息之外的属性，在建模中，它是表面可视属性的结合，它直接影响到渲染时物体表面的光泽与观察效果。

```
  //设置一个球体，这里仅是众多形状的一种。
  //如果需要其他几何体，也可以写其他代码构建特殊几何体
  var round;
  function initObject(){
    round = new THREE.Mesh(
          //材质、颜色设定
          new THREE.MeshLambertMaterial({color: 0xCCCCCC})
        //宽度、高度设定
          new THREE.SphereGeometry(22,22),
    );
    scene.add(round);
    sphere.position.set(0,0,0);
  }
```

2.5 光源设置

光源是影响三维物体浏览的一个重要因素。在不同强度，不同方向的光源下，相同的材质在渲染后会呈现出不同的显示效果。一个场景中可以设置多个光源。

```
  //设置光源
  var light;
  function initLight() {
    light = new THREE.DirectionalLight(0xcc0000, 1.0, 0);//设置平行光源
    light.position.set( 222, 222, 222 );//光源向量
    scene.add(light);//  将这个光源添加到场景中
  }
```

2.6 图像渲染

三维图像渲染，是一种将图形着色并方便用户观察的技术。显示器在没有渲染时是看不到任何图像模型的。渲染器通过照相机获得了需要的渲染范围，计算光源对物体的影响。通过光源、物体材质、物体形状、阴影等多个参数的影响，获得几何、视图方向、光源以及纹理信息。渲染完成以后，应用软件最终获得二维图形，并展示在屏幕上。

```
//开启 Three.js 渲染器
//声明全局变量（对象）
var renderer;
function initThree() {
    //用 document 得到节点的宽度和高度
    width = document.getElementById('canvas3d').clientWidth;
    height = document.getElementById('canvas3d').clientHeight;
    //生成渲染器对象，并且将抗锯齿效果为设置有效
    renderer=new THREE.WebGLRenderer({antialias:true});
    //将渲染器和画布的大小调成一致
    renderer.setSize(width, height );
    //增加一个新的元素到 canvas3D 中去
    document.getElementById('canvas3d').appendChild (renderer.domElement);
    //背景色修改(clearColor)，其中第二个参数是透明度
    renderer.setClearColorHex(0xEEEEEE, 1.0);
}
```

测试以上程序，浏览器窗口中绘制出一个球形模型：

图 3 三维图形展示

这是一个在浏览器中进行的完整 3D 建模过程。以此为通用的基础代码进行建模，在建模时只需要修改形状与材质部分的代码，浏览器就可以在动态创建并向用户显示相应的三维模型。

3 系统分析

3.1 文件格式转换分析

在上述文件转换过程中，实现该功能的关键步骤是提取三维模型的格式并构建新文件。为了实现上述提到的文件格式转换功能，需要了解文件格式转换的具体方法。

对于任何一个 3D 模型来说，其构成元素都是顶点、边和面 3 种图形。为了完整表示几何体的信息，在文件中需要包括几何数据和拓扑信息两方面，二者缺一不可。[16]

（1）几何数据：几何数据能确定三维形体在欧式空间中的位置与大小，比如顶点v的坐标(x,y,z)、边的方向矢量(e_x,e_y,e_z)的法向量(n_x,n_y,n_z)。点、边、面各个元素之间可以互相转换。

（2）拓扑信息表示三维形体的构造方法，即在该三维物体中各个元素之间的相互关系。如层次关系、连接关系等，都是属于拓扑信息方面的内容。

以常用三维模型文件格式 STL 的文件源码为例，该文件的通用数据格式如下：

```
solid filenamestl //文件路径及文件名
facet normal a1 b1 c1 // a1,b1,c1 为法向量三角面片在坐标轴上的分量
    outer loop
    vertex x1 y1 z1 // 三角面片第一个顶点的坐标
    vertex x2 y2 z2 // 第二个顶点坐标
    vertex x3 y3 z3 // 第三个顶点坐标
    endloop
    endfacet  // 第一个三角面片定义完毕
    ……
   endsolid filenamestl // 文件结束 [17]
```

设该几何体数据信息中含有 n 个三角面片的法向量值，则该文件中有 3n 个几何体的顶点。设$v_{3n-2}(x_{3n-2},y_{3n-2},z_{3n-2})$，$v_{3n-1}(x_{3n-1},y_{3n-1},z_{3n-1})$和$v_{3n}(x_{3n},y_{3n},z_{3n}$分别是第 n 个三角面片三个顶点的坐标，

集合

$$V=\{\ v_1(x_1,y_1,z_1),v_2(x_2,y_2,z_2)\ldots\ldots,v_{3n}(x_{3n},y_{3n},z_{3n})\ \}$$

是包含该几何体所有顶点的点集。

集合

$$P=\{\ p_1(a_1,b_1,c_1),p_2(a_2,b_2,c_2)\ldots\ldots,p_n(a_n,b_n,c_n)\ \}$$

是包含该几何体所有法向量的向量集。

从根据集合和三角面片的个数为 n，可推出

集合 $E=\{\ e_{3n-2}(|x_{3n-1}-x_{3n-2}|,|y_{3n-1}-y_{3n-2}|,|z_{3n-1}-z_{3n-2}|)\ ,e_{3n-1}(|x_{3n}-x_{3n-1}|,|y_{3n}-y_{3n-1}|,|z_{3n}-z_{3n-1}|),e_3(|x_{3n}-x_{3n-2}|,|y_{3n}-y_{3n-2}|,|z_{3n}-z_{3n-2}|)\}$

是包含该几何体所有边的边集。

3.1.1 从 STL 转换到 Three.js 图形显示的方法

在 Three.js 中，可以使用以下集合数据来定义 3D 模型的文件信息：

①$v_1(x_1,y_1,z_1),v_2(x_2,y_2,z_2)\ldots\ldots,v_{3n}(x_{3n},y_{3n},z_{3n})$

其中每个三元组(x_k,y_k,z_k)表示一个顶点坐标值。

②$p_1(a_1,b_1,c_1),p_2(a_2,b_2,c_2)\ldots\ldots,p_n(a_n,b_n,c_n)$

其中每个三元组(a_n,b_n,c_n)表示第 n 个三角面片的法向量。

而□和□的数据信息在 STL 源文件格式中均已给出。

因此，使用 Three.js 编写的通用 STL 文件格式的代码如下：

```
glColor3ub(192,192.192); //默认网格颜色为灰色
//构造一个全部由三角形面组合形成的三维模型
geBegin(GL_TRIANGLES);
//num 为网格中的三角形数目
//a[n],b[n],c[n]为三角面片的法向量在 x,y,z 轴上的分量
For(int I = 0; I < n; i++){
        Traiangle arr; //一个三角形的顶点数组
        Float t[3];       //三角面片 tn 的法向量
        t[0] = a[n];
        t[1] = b[n];
        t[2] = c[n];      //给法向量的三个分坐标赋值
        glNormal3fv(t); //给定法向量
        glVertex3fv(arr.x1); //给定第一个顶点
        glVertex3fv(arr.y1); //给定第二个顶点
        glVertex3fv(arr.z1); //给定第三个顶点
}
glEnd(); //构造结束
```

对任意 STL 格式的文件，均可从源文件中提取出它的点集和 V 向量集P，将其作为参数，代入通用 STL 文件格式的代码中，即可将文件转成在浏览器上显示。

3.1.2 从 STL 转换为其他文件格式的方法

为了将一个文件格式转换为另一个文件格式，需要按照其中一种文件格式的标准，将其中包含的几何体的几何数据和转换到另一个文件格式中去。

以 STL 向 AMF 格式转换为例，现对 STL 转换为 AMF 的格式方式进行分析：[15]

AMF 文件的通用文件格式如下：

```
<?xml version="1.0"?>
<AMF>
<Object PrintID = "0" units = "mm">
<object id="1">
<Mesh>
<Vertices>
    <Vertex VertexID="0">
    <VertexLocation x="x1" y="y1" z="z1"/>
    </Vertex>
    <Vertex VertexID="1">
```

```
    <VertexLocation x="x2" y="y2" z="z2"/>
    </Vertex>
    ...
      </Vertices>
    <Region FillMaterialID = "0">
    <Triangle V1 = "0" V2 = "1" V3 = "2"/>
    <Triangle V1 = "2" V2 = "1" V3 = "4"/>
        ...
  </Region>
</Mesh>
</Object>
</AMF> [18]
```

在 AMF 文件中，所有的数据信息以 XML 格式存储，其中<Vertices>标签中包含了该三维模型文件中的所有边向量。若该文件代表的三维模型由 n 个三角面片构成，则原文件中有 3n 个<vertex>标签。

由<Vertex VertexID="3n">
<VertexLocation x="x3[n]" y="y[3n]" z="z[3n]"/> 可知：
<vertex>中包含的 x,y,z 参数即为边e的三个坐标分量。所以对每个<vertex VertexID = "n">，都可建立相应映射，将向量$e_n(x_n, y_n, z_n)$转换为

```
<Vertex VertexID="n">
     <VertexLocation x="x[n]" y="y[n]" z="z[n]"/>
</Vertex>
```

而 AMF 文件中共有 3n 个<vertex>标签。

因此，源文件中所有的边向量均可代表边集集合中的相应元素。

在<Region>标签中含有<Triangle>标签，其中的 V1,V2，V3 即为一个三角面片的三条边向量。

由<Triangle V1 = "n[1]" V2 = "n[2]" V3 = "n[3]"/>可知，该三角面片的三条边向量分别为e_{n1}、e_{n2}和e_{n3}。可根据三条边向量的数据，求出该三角面片的顶点v_{n1}、v_{n2}和v_{n3}。因为几何体数据中有 3n 个几何体的顶点，n 个三角面片，所以点集V中所有元素落在三角面片的顶点上，因此，对每个<triangle>均可建立相应映射，将其三条边的交点转换为点集V中的相应元素。

所以根据源文件格式中的三角面片的所有顶点，可求出该几何体点集V中的所有元素。

使用相同的分析方法，可以分析出.obj 文件,.dwg 文件等多种文件互相转换，以及各种文件转换到 JavaScript 的映射方法，从而实现在线显示的功能。

3.2 系统结构设计

一个典型的静态三维模型中包含几何体顶点，贴图

坐标点，法线、面、曲线等信息[12]，若将三维模型文件存储到后端，则网站数据库中将包含大量信息，会造成服务器数据冗余。该平台主要实现“轻量化”的浏览与文件转换功能，因此需要摆脱大量数据造成的负担。因此，放弃使用数据库不仅能够减轻数据造成的压力，同时也方便读取各种不同的文件，将其转换成 JavaScript 可识别的格式。

用户在导入相关 3D 文件时，该模块对文件进行相应格式转换，成为 JS 可执行代码，然后浏览器 JavaScript 层使用 WebGL 和 Three.js 框架将文件转换成浏览器可解析的数据，浏览器通过三维图形显示技术将其映射到显示器上，形成 3D 图像。同时，浏览器视图层向后台发出请求，服务器将接收到的文件进行相关处理，然后为前端提供文件导出的接口。使用该系统结构，不仅避免了数据库的使用给服务器带来的瓶颈，也充分利用了 JavaScript 层强大的脚本解析能力，充分发挥了浏览器前端的运算能力，服务器可以轻巧地处理相关文件转换模块事件，大幅提升文件格式转换速度

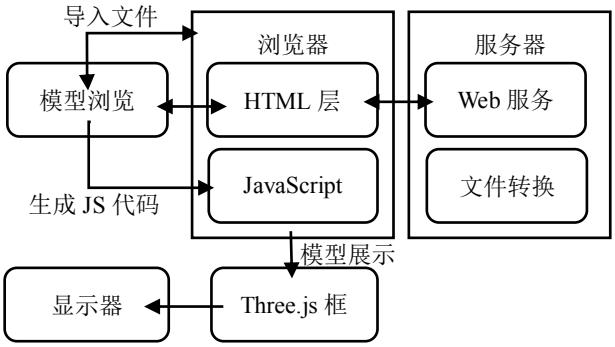

图 4 3D Web 系统结构框架

3.3 系统设计特点

从图 3 中可以看出，该应用程序可以完成两个任务：第一是实现了 Web 3D 模型的可视化。在这个任务中，所有的流程均与后端服务器无关。第二是实现了各种三维文件格式的转换功能。该部分功能主要由后端服务器完成。这种前端与后端分离的设计方法让整个架构的思路更加清晰，不同模块之间各司其职，提高了整个应用软件的运作效率，不仅加快了前端的显示速度，也把主要的文件计算任务交给服务器。将二者任务分离，便于以此为基础进行二次开发。

4 设计难点

4.1 展示界面渲染器的编写

渲染器是 3D 引擎最重要的核心，它完成的任务是将 3D 模型绘制到屏幕上，其作用是翻译实时场景模型文件，根据相应代码规则，实时渲染模型文件，在客户端实时地显示模型文件。[13]渲染过程是向用户展示图像过程中最关键的技术。在渲染中，渲染引擎需要对物体的形状，纹理，照明等信息进行精密计算，最终得到模型图像。三维计算机图形的渲染过程中，计算量非常大，需要耗费较长时间才能完成。WebGL 和 Three.js 均已有相关预置好的渲染引擎，可以高效且简洁地完成三维模型的渲染任务。

4.2 对加载速度的要求

为了推进轻量化的浏览体验，在设计应用程序时显然需要考虑到加载时间的长短。因此对加载速度的要求是另一个难点。影响加载时间的因素主要有两个：第一个因素是模型渲染时间，第二个因素在于外部导入模型时服务器加载文件的时间长短。

对于第一个因素来说，采用高质量的渲染器和减少模型的复杂度有利于缩短渲染时间；

对于第二个因素来说，文件导入时的加载时间和文件解析速度则是缩短时间的关键，因此，在文件转换模块中，使用高效的解析算法可以有效地提高加载速度。

4.3 交互式设计

在 JavaScript 中，做在三维场景中实现拖放式设计的难点在于，鼠标位置信息是二维的，而场景内对象的位置是三维的，二者不存在一一对应关系。[14]用户在线浏览 3D 模型文件时，

总是需要从多个视角观察物体。在传统的图形设计软件中，用户可以切换不同的视图来达到多方位观察的目的。而在网页架构中，使用鼠标推拉和拖动的方式能更方便用户的使用。因此，有必要定义一些鼠标事件来提供转换视角的方法，如模型的缩放，旋转，内切等。这些事件涉及到使用 JavaScript 与渲染完成后的场景页面交互。

5 应用效果与分析

5.1 应用效果

笔者实现了 STL 与 AMF 文件的格式转换与在线读取功能，将其部署到服务器上。经过测试，模型展示消耗时间和文件转换时间如下：

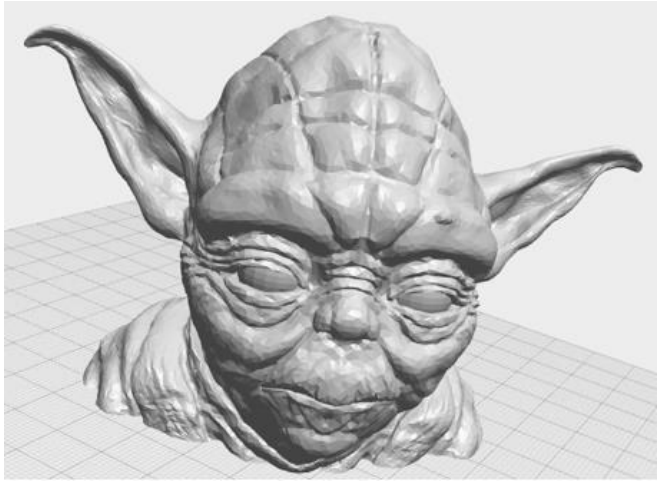

图 5 STL 文件模型

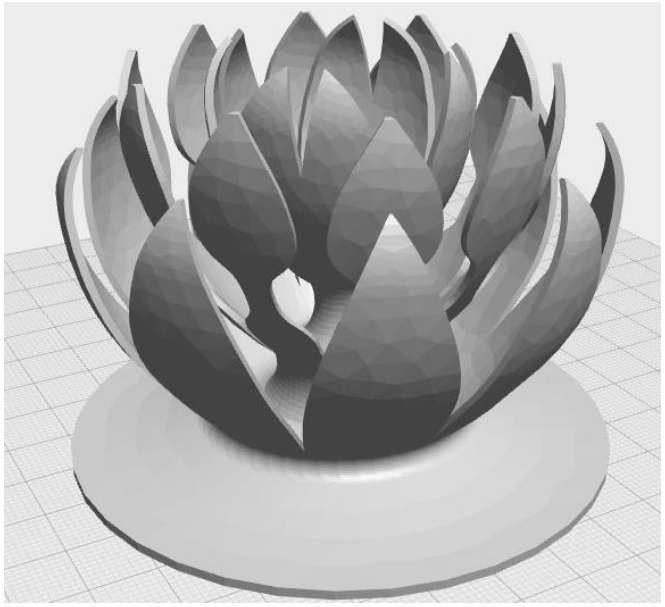

图 6 AMF 文件模型

表 2 AMF、STL 文件从读取到渲染到界面上的时间

文件体积(KB)	AMF 格式(ms)	STL 格式(ms)
44	11	304
82	42	442
244	122	902
398	173	1224
766	332	2072
1997	682	4968

表 3 AMF、STL 文件格式转换时间

文件体积(KB)	AMF->STL(ms)	STL->AMF(ms)
44	103	97
82	152	142
244	301	288
398	488	437
766	907	920
1997	2540	2260

5.2 应用分析

评价软件好坏的指标一般从两个方面评价：第一方面是软件是否能够成功执行并完成预期的功能；第二方面是软件执行核心业务时的时间效率。对于本软件来说，衡量该软件是否成功的标准在于在线浏览功能和文件转换功能。

从在线浏览的角度看，实验表明，该软件能够在较短时间内在线渲染出高质量的模型，模型精度如图 4、图 5 所示。对于 amf 格式，该系统能在 1s 以内渲染完成并展示出 2000KB 以下的所有模型文件；对于 stl 文件格式来说，该系统能在 5s 以内渲染出 2000kb 以下的模型文件，因此该设计方案可行，并有良好的使用价值。

从文件转换的角度看，笔者进行了多组测试，被测的 AMF 文件和 STL 文件在该系统中均可实现互相转换，且转换后的文件均能够被 3ds MAX、Maya、Lightwave 等三维模型设计软件成功读取。对于所有大小为 2000kb 以下的模型文件，系统均能够在 3s 内完成相应的文件格式转换。因此，该功能对于普通用户来说，拥有推广的价值与潜力，实现了良好的应用效果。

6 结语

本文针对目前 3D 模型文件在日常使用的不便之处，提出了一种构建在线浏览、文件格式转换的应用软件，并说明了其构建原理和技术方法。在软件中对格式转换和前端展示功能均做出优化，增强了应用的性能，使用户体验得到提升。

目前 Web 3D 文件格式在领域内尚未统一，各种新技术与新方法层出不穷。虽然三维图像处理在网络化进程中尚有未解难题，但部分技术已经取得相当的成果。[2]

随着 HTML5 以及 JavaScript 框架等技术的广泛使用，网络技术和硬件设施的发展，网络三维化将会成为今后网络多媒

体发展方向的主流，Web 3D 的普及将不再遥远。

参 考 文 献

[1] 蒋燕萍，夏旺盛，黄心渊．几种Web3D技术的比较[J]．北方工业大学学报，2003，01：21-25．

[2] 韩义．Web3D及Web三维可视化新发展——以WebGL和O3D为例[J]．科技广场，2010，05：81-86．

[3] Khronos: WebGL-OpenGL ES 2.0 for the Web. [EB/OL]. https://www.khronos.org/webgl/ . Khoronos.org, 2016.4.3.

[4] Wikipedia: WebGL(Web Graphics Library).[EB/OL]. https://en.wikipedia.org/wiki/WebGL . Wikipedia, 2016.3.31.

[5] 黄健．面向通用性的Web3D虚拟展览馆的设计与实现[D]．电子科技大学，2012．

[6] Mr Doob: Three. jshttps://github.com/mrdoob/three.js[EB/OL]Mr Doob 2016.4.2.

[7] 王磊，高珏，金野，许华虎．基于Web3D无插件的三维模型展示的研究[J]．计算机技术与发展，2015，04：217-220．

[8] 赵孔阳，朱军，尹灵芝．基于WebGL的虚拟场景网络漫游研究[J]．测绘与空间地理信息，2015，12：40-43．

[9] 卞敏捷，高珏，高洪皓，许杰品．Web3D可视化技术的研究与应用[J]．计算机技术与发展，2015，06：141-144．

[10] 郑华，宿景芳．面向Web的三维模型生成与处理技术[J]．现代电子技术，2015，24：83-86．

[11] Three.js documentation http://threejs.org/docs/index.html [EB/OL] Mr Doob 2016.4.2

[12] 何龙，杜鹏．基于WebGL的三维可视化[J]．科技资讯，2015，30：23+25．

[13] 刘运增．互联网上的三维技术：Web3D[J]．计算机与网络，2003，10：32-35．

[14] 林绮屏．基于Web3D技术的网络三维展示系统的设计与实现[J]．情报科学，2011，05：692-695+699

[15] Jonathan D. Hiller, STL 2.0: A proposal for a universal multi-material additive manufacturing file format. [J]. Solid Freeform Fabrication Symposium (SFF'09), Aug 3-5 2009, Austin, TX, USA.

[16] 周莉，苏鸿根．通用3D模型文件格式和算法的研究及其OpenGL实现[J]．计算机工程与设计，2009，02：433-436+439．

[17] STL (file format) https://en.wikipedia.org/wiki/STL_(file_format) [EB/OL] Wikipedia, 2016.3.31.

[18] Additive Manufacturing File Format https://en.wikipedia.org/wiki/Additive_Manufacturing_File_Format [EB/OL] Wikipedia, 2016.3.31.

作者简介：童凌(1994 -)，男，浙江丽水人，南华大学计算机科学与技术学院本科生，主要研究领域为软件工程与 Web 开发，身份证号为 33250199406010413，手机为 13787730786，单位为南华大学，地址为湖南省衡阳市蒸湘区南华大学计算机科学与技术学院，邮编号码 42101，E-mail 为 545716730@163.com